

Akshat — MLAAD, IndicSynth, Hindi clips

RTX 4060 · you own generator diversity and the clips that win the demo

Read this first

Where you are, and what changed

Your environment work was correct and none of it is wasted. You installed ffmpeg via conda, got torchcodec loading, confirmed CUDA and the 4060 — all good, keep it. **The hang at Bengali: streaming ... was a bug in our script, not anything you did.**

What was wrong: it used `datasets` streaming mode, which pulls an entire parquet row group before yielding a single row. IndicSynth stores audio bytes inside the rows, so one row group is hundreds of megabytes — the script sat there downloading in silence with no output and looked frozen. Both of your Ctrl+C's landed inside `pyarrow.to_batches()`, which is exactly where that blocking read happens.

The rewritten version downloads whole shards with a visible progress bar and reads the audio directly. **It no longer uses torchcodec at all**, so that entire `libtorchcodec_core*.dll` problem is now irrelevant to this task.

```
cd /d/SONIX
git pull
```

ONE QUICK WIN BEFORE YOU START

`nvidia-smi` shows **LM Studio holding your GPU**. Close it before extraction — you want all 8 GB, and it is the difference between `--batch 8` working and hitting CUDA out of memory.

Your two jobs

And which one actually matters

#	JOB	STATUS	TIME
1	IndicSynth → embeddings	Unblocked, not started	~1h download + ~1h GPU
2	Hindi real + cloned voice pairs	Not started	2-3h of your own time
3	In-the-Wild	Only if 1 and 2 are done	~2h

JOB 2 IS THE MORE IMPORTANT ONE. START IT TODAY.

Job 1 is a download — it runs unattended while you do other things, and if it fails we just run it again. **Job 2 cannot be re-run at 2am on the day.** It needs real people speaking into real phones, and it is the only thing on this entire project that can tell us whether the fix worked. No dataset substitutes for it.

Kick off Job 1's download, then spend your attention on Job 2 while it runs.

Job 1

IndicSynth — four steps

IndicSynth is synthetic speech in 12 Indian languages: Bengali, Gujarati, Hindi, Kannada, Malayalam, Marathi, Odia, Punjabi, Sanskrit, Tamil, Telugu, Urdu. Its generator is `freevc24` — *voice conversion*, a different family from the TTS models Navya is fetching. Two families of fake beat twice as much of one.

Step 1 — download (~16,800 clips)

```
python prep_indicsynth.py --list
python prep_indicsynth.py --per-lang 1400 --out data/indicsynth
```

You should now see, per language:

```
Hindi: need 1400 clips
  downloading Hindi/train-00000-of-000NN.parquet ...
    200
    400
Hindi: 1400 clips
```

If it prints nothing for more than a couple of minutes, that is wrong — tell Yugal. Silence is the symptom of the old bug.

WHY 1400 PER LANGUAGE AND NOT "ALL OF IT"

The subsets are wildly uneven — Sanskrit has 378k rows, Malayalam 34k. Take it whole and one language supplies most of the training data, and the model learns *that language* instead of learning synthesis. Even coverage beats volume every time here.

GATE 1 — CHECK THE SPREAD BEFORE SPENDING GPU TIME

The script prints a per-language table at the end. You want **roughly equal counts across most of the 12 languages**. If only two or three produced clips, stop and send Yugal that table — extracting a lopsided set wastes an hour and gives us data we cannot use.

Step 2 — extract embeddings (GPU, ~1 hour)

```
python src/extract_embeddings.py --split train --audio-dir data/indicsynth \
  --out outputs/embeddings_indicsynth --batch 8
```

CUDA out of memory → `--batch 4`, then `--batch 2`. Check progress any time from another terminal:

```
ls outputs/embeddings_indicsynth/train/shard_*.npy | grep -v labels | wc -l
```

Step 3 — stamp the labels. Do not skip this.

```
python stamp_labels.py --emb-dir outputs/embeddings_indicsynth/train --label 1
```

`--audio-dir` extraction marks **every row label 0 (real)** by default. Every one of these clips is a fake. Without this step they train as genuine speech.

Step 4 — verify, and actually read the output

```
python stamp_labels.py --emb-dir outputs/embeddings_indicsynth/train --check
```

GATE 2 – EVERY ROW MUST READ SPOOF (1)

If it shows any bonafide rows, **stop**. Training on unstamped fakes teaches the model that synthetic Hindi is genuine — which is the exact bug we are trying to fix, made worse. This is a silent failure: nothing else will warn you.

(stamp_labels.py itself was broken until 2 Sept — it never once successfully wrote a label. Navya found and fixed it. You have the fix if you pulled.)

Send Yugal

- The folder outputs/embeddings_indicsynth/
- The per-language count table from step 1
- The full --check output from step 4, pasted

Job 2

Hindi real and cloned pairs — the one that decides everything

Every clip you delivered last round was English — check clips_manifest.csv , every row says so. That is a large part of why nobody caught this bug until yesterday: we had no Indian-language evidence at all.

What the bug is, so you know what you are testing

Our detector was trained on English, then we added a large amount of genuine Indian speech and no fake Indian speech. So the only thing it could learn about Indian audio was “this is real”. Given a cloned Amitabh Bachchan clip, it says **real**. Your recordings are how we prove whether the new model fixes that.

What to collect

TYPE	TARGET	NOTES
Real Hindi	30+	Different phones, laptops, quiet room, noisy room, speakerphone, earphone mic. Include short (2-5s) and quiet ones.
Cloned Hindi	15-20	ElevenLabs, Play.ht or similar — the same class of tool a real scammer uses.
Matched pairs	as many as possible	The same person, recorded AND cloned. This is the important part.

Why matched pairs matter so much: if a real clip and a cloned clip come from different people, and the model gets them right, we cannot tell whether it detected the *synthesis* or just preferred one voice. Same speaker on both sides removes that doubt entirely — and it is by far the most convincing thing to show a judge.

Use family-emergency and “send money now” scripts so it matches our impact story. For three or four, play the cloned audio out of a laptop speaker and re-record it on a phone — that is what a scam call actually sounds like down a real line.

Folder layout — the folder is the label

```
sonix_hindi/  
  real/      <- genuine human recordings  
  fake/      <- cloned
```

Nothing to match up by hand, nothing to get wrong. Fill in `clips_manifest.csv` as `filename,label,language,source,device` — and put **Hindi** in the language column this time. For matched pairs, use names that pair up: `hindi_real_ravi_01.wav` and `hindi_fake_ravi_01.wav`.

THESE ARE TEST DATA. DO NOT EXTRACT THEM, DO NOT STAMP THEM.

Send Yugal the audio folders and the manifest, and stop there. They are how we *prove* the fix worked. Put them in training and the proof evaporates — same rule as In-the-Wild and DF21.

Job 3

In-the-Wild — only after 1 and 2

```
hf download mueller91/In-The-Wild --repo-type dataset --local-dir data/in_the_wild
```

Note it is `hf` now, not `huggingface-cli` — that command was deprecated and no longer runs.

Verify both classes exist before extracting:

```
python -c "import pandas as pd,glob; f=glob.glob('data/in_the_wild/**/*.meta.csv',recursive=True)[0];  
print(f); d=pd.read_csv(f); print(d.shape); print(d.iloc[:,-1].value_counts())"
```

```
python src/extract_embeddings.py --split itw --batch 8 --out outputs/embeddings \  
--itw-root data/in_the_wild
```

No label stamping for this one — the extractor reads `meta.csv` directly and handles the `bona-fide` vs `bonafide` spelling difference itself.

BEFORE YOU LOOK AT THE IN-THE-WILD NUMBER

It will be far worse than our 1.49%, and that is expected, correct, and not your fault — it uses synthesis tools we never trained on. **10-30% EER is the normal, publishable result for this benchmark.** Around 40% suggests a label problem; around 0% means something leaked. Do not spend a night trying to “fix” a correct-but-ugly number — reporting it honestly is what makes us look like researchers rather than people overselling. And never train on it.

Report back

```
IndicSynth : ____ clips across ____ languages  
             per-language table pasted  
             stamp --check output pasted in full  
  
Hindi clips: ____ real / ____ cloned / ____ MATCHED pairs  
             (matched = same speaker appears in both folders)  
  
In-the-Wild: ____ bonafide / ____ spoof, EER ____% (only if reached)
```

Troubleshooting

SYMPTOM	CAUSE / FIX
stuck at "<lang>: streaming ..."	Old script. <code>git pull</code> — the new one prints "downloading <shard>"
ImportError: install 'torchcodec'	Old script. The new one does not use torchcodec at all
Could not load libtorchcodec	Same — no longer on the path for this task
huggingface-cli is deprecated	Use <code>hf</code>
Invalid value. Repo id must use...	You literally passed <code>...</code> as the repo name
CUDA out of memory	Close LM Studio, then <code>--batch 4</code>
only 2–3 languages produced clips	Stop, send Yugal the <code>--list</code> and per-language output
check shows bonafide rows	Stop. Re-run the stamp; if it persists, tell Yugal
no .wav files directly in ...	Point <code>--audio-dir</code> at the folder holding the files directly
symlink warning on Windows	Harmless. Uses more disk, works fine

When in doubt, stop and ask Yugal. A wrong number is far worse than a missing one, and a mislabelled dataset is worse than no dataset.

The situation

What broke, and why

Our detector fails on Indian speech, and we now know exactly why.

Every model up to `head_robust_v2` was trained on ASVspoof 2019 — English studio speech — plus augmentations that vary *rooms, noise and phone channels*. None of them vary *language*. So on 42,082 genuine IndicVoices clips, the baseline called **56% fake** and `robust_v2` called **58.6% fake**. That is why the Modi and Bachchan clips failed.

Navya then added IndicVoices to training and Indian false alarms collapsed to **0.02%**. Real fix? No — a second bug. IndicVoices is **bonafide-only**: every one of its ~37,000 rows is labelled real. So the only thing the model could learn from Indian audio was “*Indian phonetics* → *real*”. Tested on a cloned Amitabh Bachchan clip, `head_full_ho` calls it **real**.

MODEL	REAL BACHCHAN	CLONED BACHCHAN	WHAT IT ACTUALLY LEARNED
head_robust_v2	fake ✗	fake ✓	“Indian audio = fake”
head_full_ho	real ✓	real ✗	“Indian audio = real”

Neither is detecting anything. Each gets one column right by accident.

THE ROOT CAUSE, IN ONE SENTENCE

As long as every Indian-language sample in training is labelled real, the language itself is a shortcut to “real”, and a cloned Indian voice walks straight through. No threshold, augmentation or calibration fixes this.

The fix

Put Indian languages on **both sides of the label**, from **several different synthesis families**. Then the model cannot use language or accent as a cue — it has to find real synthesis artefacts or nothing.

DATASET	SYNTHESIS FAMILY	OWNER
Indic-CodecFake	Neural audio codecs — 8 of them (DAC, Encodec, SoundStream, SpeechTokenizer, FunCodec, AudioDec, SNAC, MIMI)	Navya
MLAAD (Indic slice)	Text-to-speech — up to 140 models	Akshat
IndicSynth	Voice conversion (freevc24)	Akshat
IndicVoices <code>have it</code>	Real human speech (bonafide side)	Navya

Three *families*, not just more hours. A head trained on one generator learns that generator — which is precisely the trap we fell into with English-only ASVspoof 2019. Generator diversity is what buys generalisation.

How we train

One model, many ablations

Two questions came up, so here they are answered for everyone.

Is the new model built from scratch, or on top of the latest one?

Neither, exactly — and this matters. Our pipeline has two stages:

STAGE	COST	REUSABLE?
audio → embeddings (frozen wav2vec2 XLS-R, 300M params)	GPU, hours	Yes — forever, per dataset
embeddings → head (MLP, 262,657 params)	CPU/GPU, minutes	Cheap to redo

So we never “continue training” `head_full_ho`. We retrain a **fresh head on the union of every embedding root** — the old ones plus the new ones. That is what `--extra-emb-root` does, and it stacks (it uses `action="append"`, verified).

Fine-tuning the existing head on only the new data would be actively wrong: it would forget the English attacks it can currently catch.

One model or several?

One production model, plus four throwaway ablation heads for the results slide. Once the embeddings exist, each head takes minutes, so the ablations are nearly free — and they are what let us say “*we measured which data helped*” instead of “*we threw everything in and hoped*”.

The expensive, parallel part is extraction. That is why it is split three ways across our GPUs. Training is serial and Yugal does it.

THE RULE THAT MAKES OR BREAKS THIS

Cap clips per (language × generator). If one TTS model contributes 40,000 clips and another contributes 200, the head learns to detect the loud one. Every extraction step below has a cap flag. Use it.

Hard rules

What must never enter training

These are test sets. Training on any of them destroys the numbers that prove we generalise, and a judge who spotted it would be right to dismiss the project.

- **ASVspoof 2021 DF** — our cross-dataset headline
- **In-the-Wild** — the honest generalisation test, named on slide 3
- **The IndicVoices holdout** (`embeddings_indicvoices_ho`) — 12% held out by recording
- **Akshat's 20 labelled clips** and **the Bachchan pair** — our real-world evidence

Everything else — ASVspoof 2019 train, the augmented copies, IndicVoices train split, Indic-CodecFake, MLAAD, IndicSynth — is fair training data.

SECOND HARD RULE — LABEL STAMPING

`extract_embeddings.py --audio-dir` marks **everything label 0 (real)** by default. Every spoof folder must be stamped `--label 1` afterwards, and verified with `--check`. Skip it and your fakes train as genuine — a silent, catastrophic bug that nothing warns you about.

`stamp_labels.py` was broken until 2 Sept (it never wrote a label successfully). Make sure you have pulled since then, and always read the `--check` output rather than assuming.

Sections after your own tasks are identical in all three handbooks, so nobody has to read someone else's. Datasets: Indic-CodecFake (helixometry.github.io/IndicFake) · MLAAD (deepfake-total.com/mlaad) · IndicSynth (huggingface.co/datasets/vdivyasharma/IndicSynth). When in doubt, stop and ask Yugal — a wrong number is far worse than a missing one.